

Introduction to Database Systems

Spring 2025

Lecture 10 - Transactions

Omar Shahbaz Khan

Transactions

Transactions

- Transactions group a set of database operations
 - Atomic, all operations succeed or none do
- May require shared database resources (rows, columns, tables)
 - Can lead to conflicts with multiple concurrent transactions
- Can be run sequentially or concurrently
 - Trade-off between consistency (data integrity) and performance (latency)

Transactions in SQL

- Use a BEGIN; COMMIT/ROLLBACK; block to group SQL statements into a transaction

```
BEGIN TRANSACTION; -- Some DBMSs only require BEGIN; to start a transaction
```

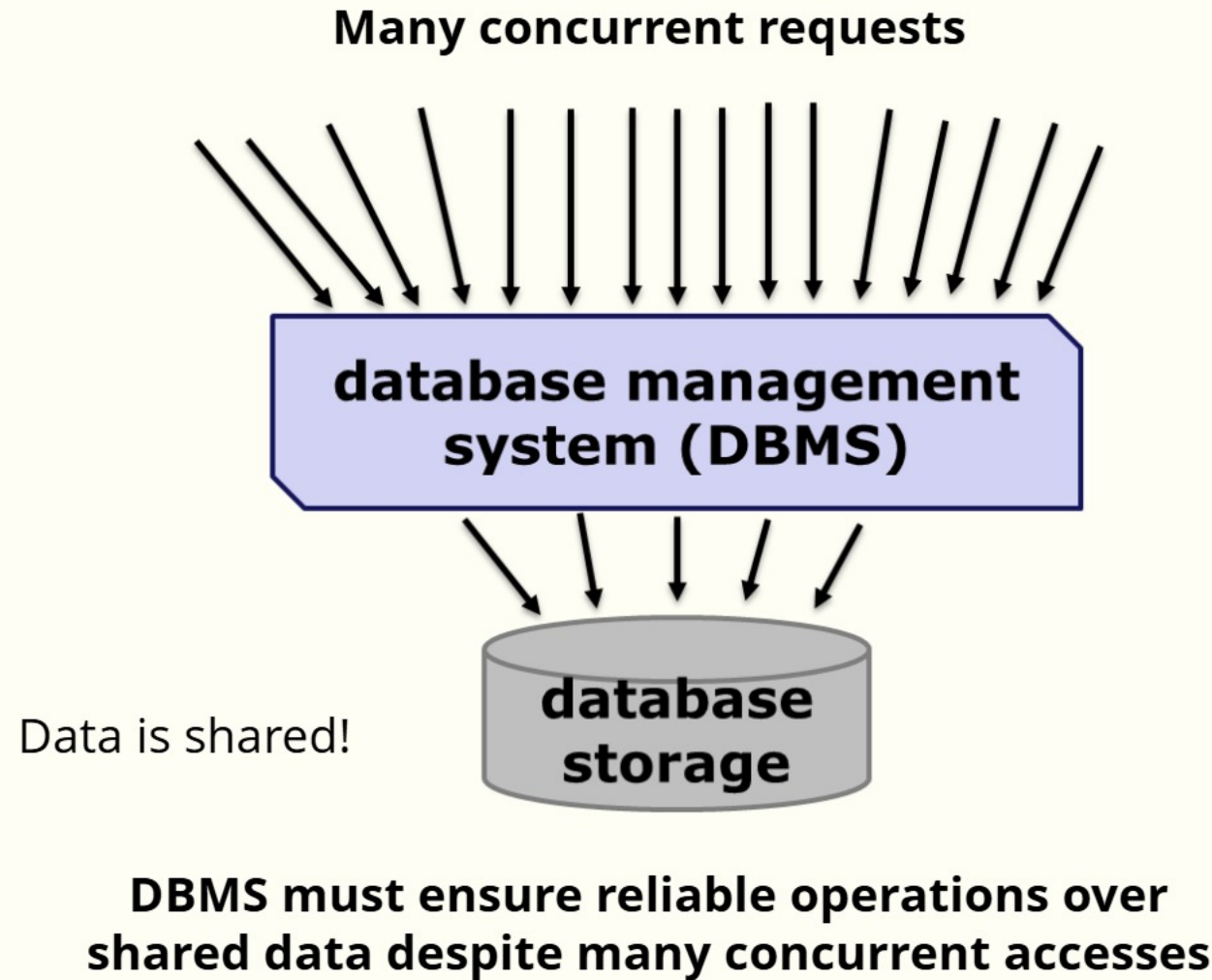
```
<SQL Statement 1>;
```

```
<SQL Statement 2>;
```

```
COMMIT; or ROLLBACK;
```

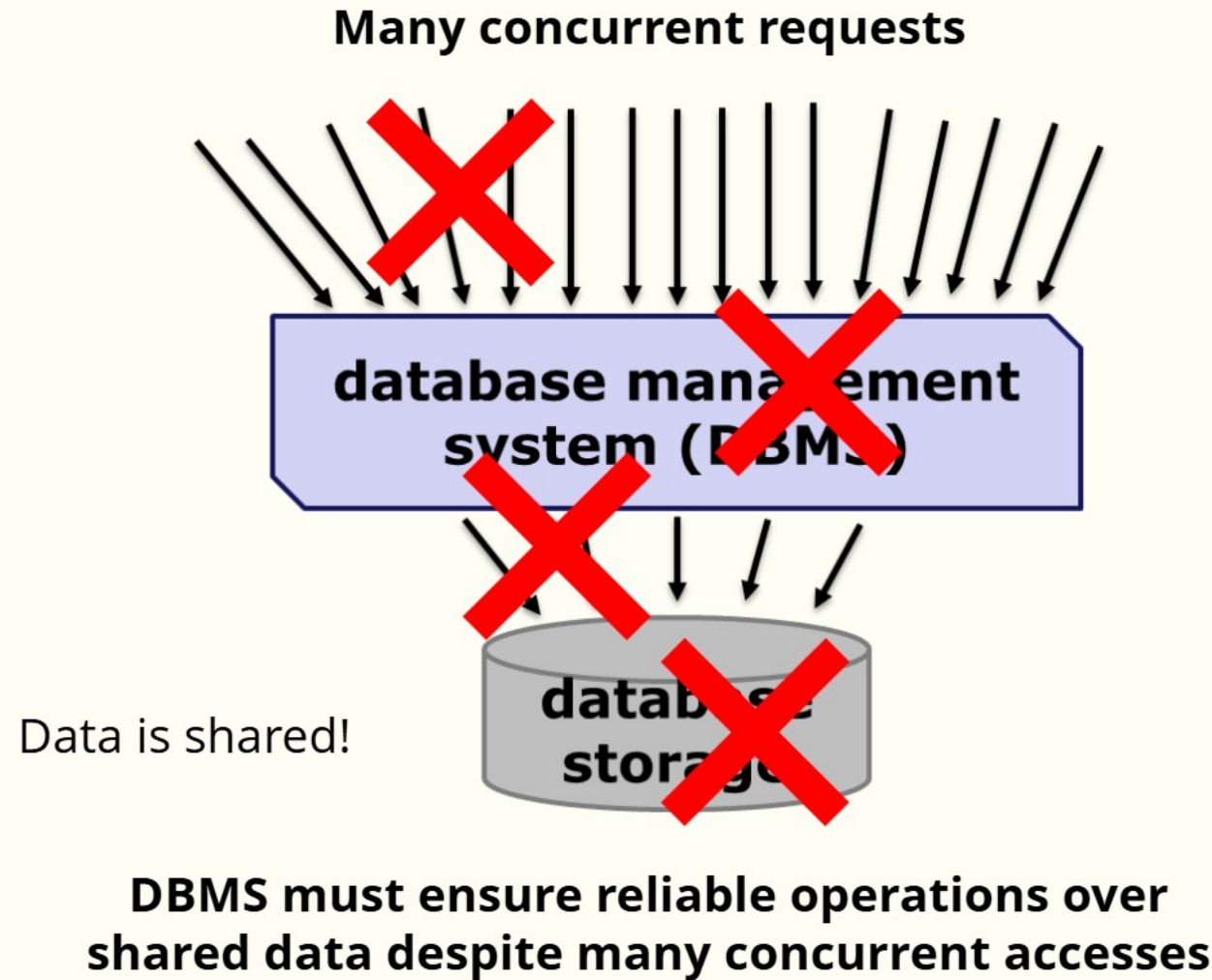
- COMMIT if the transaction runs as expected
- ROLLBACK if you encounter an error
 - This can also be used for testing
 - Make changes to the database tables, check if logic is correct, rollback to discard changes
- Save points are also available in some databases
 - Allows you to rollback state to a specific point in your running transaction
 - SAVEPOINT <sp_name>; ROLLBACK TO <name>;

Why do we need Transactions?



Why do we need Transactions?

**ANYTHING
CAN CRASH!**



To the end user everything is fine



ATOMICITY

A Transaction is
"one operation"

Each transaction
runs to completion
or has no effect at
all

CONSISTENCY

Each Transaction
moves the DB from
one consistent state
to another

After a transaction
completes, the
integrity constraints
are satisfied

ISOLATION

Each Transaction is
alone in the world

Transactions
executed in parallel
have the same effect
as if they were
executed sequentially

DURABILITY

Persistence of successful
transactions even
through system failure

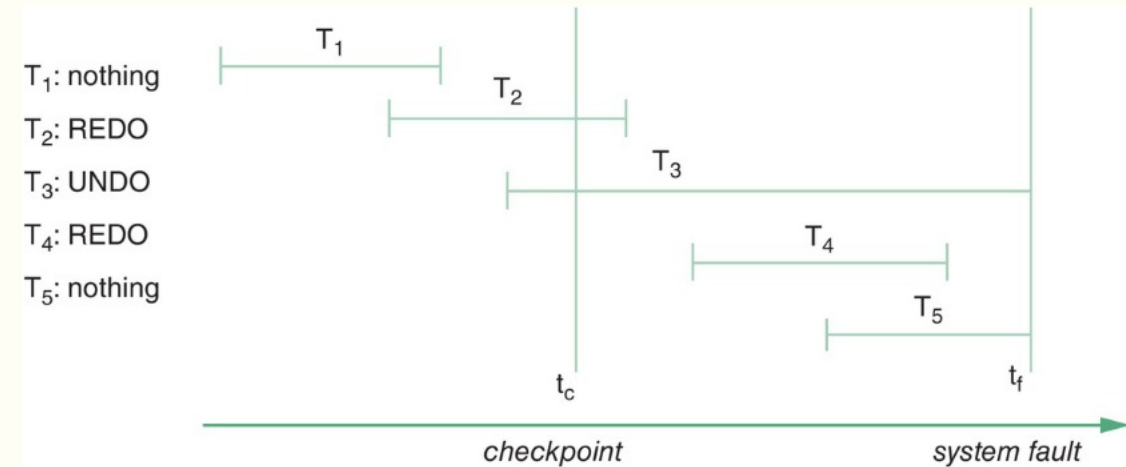
The effect of a committed
transaction remains in
the database even if the
system crashes

How to implement Transactions?

- Consistency ~= satisfying constraints
 - Use indexes for primary and foreign key, check statements, ...
- Atomicity and Durability = tracking changes
 - Logging "before" values to UNDO changes
 - Logging "after" values to REDO changes
- Isolation = preventing corrupting changes
 - Pessimistic: Locking to prevent conflicts
 - Optimistic: Time stamps to detect conflicts

Atomicity and Durability Issues

- Consider the following schedule of transactions
- Atomicity
 - Transactions abort
 - Systems crash
- Durability
 - Systems crash
- Upon restart
 - Want to see the effects of T1, T2, T4
 - Want to remove the effects of T3 and T5



Buffer Management (RAM vs Disk)

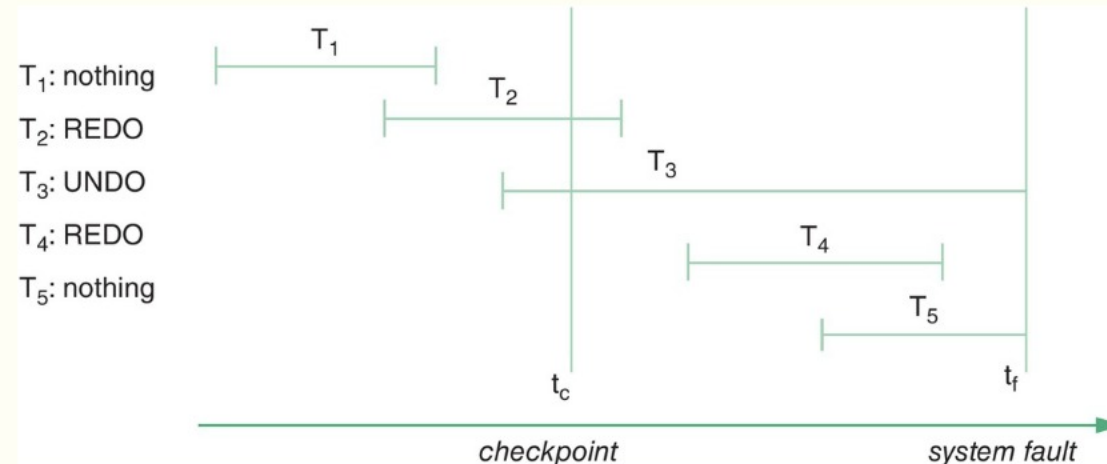
- Once a transaction completes, it performs a COMMIT
- A COMMIT does not equate to changes being on disk
- Buffer Management Policies dictate when changes are moved from RAM to Disk
 - The transaction log is always updated / written to disk
 - FORCE / NO FORCE policies affect data pages of committed transactions
 - STEAL / NO STEAL policies affect data pages of uncommitted transactions

Buffer Management Policies

- FORCE = write changed pages to disk at COMMIT
 - Ensures Durability (assuming writes are atomic)
 - Increases response time
- NO FORCE = Changed pages stay in RAM after COMMIT
 - What if the system crashes?
- STEAL = Changed pages can move to Disk before COMMIT
 - What if transaction aborts? System crashes?
- NO STEAL = Allow only updated (committed) pages to be moved to Disk
 - Ensures Atomicity (can simply discard at abort)
 - Increases response time
- We can choose one of FORCE / NO FORCE and one of STEAL / NO STEAL
 - Which combination do we pick?
 - NO FORCE / STEAL

NO FORCE / STEAL

- NO FORCE: Changed pages stay in RAM after COMMIT
 - What if system crashes?
 - Need to remember the new value of P to be able to **REDO** the changes
- STEAL: Changed pages can move to Disk before COMMIT
 - What if transaction aborts? System crashes?
 - Need to remember the old value of P to be able to **UNDO** the changes.



Write-Ahead Logging (WAL Protocol)

- Write-Ahead Logging
 1. Before any changes are written to disk
we force the corresponding log record to disk
 2. Before a transaction is committed
we force all log records for the transaction to disk
- #1 Ensures Atomicity
- #2 Ensures Durability

The Log

- Write REDO and UNDO info to log
 - Ordered list of REDO/UNDO info
 - Think of an infinite file with append only!
- Log processing must be fast - why? and how?
 - Write minimal info to log (diff)
 - <xid, pageID, offset, length, old_data, new_data> + control info
 - Many log entries per page
 - Ensure sequential writes!
 - Writing a log entry => writing all previous entries
 - **Put the log on its own disk!**

Restart Recovery

1. Analyze information about transactions from the last checkpoint
2. **REDO** the changes of committed transactions that did not make it to disk
3. **UNDO** the changes of uncommitted transactions that made it to disk

A Basic Database Setup

- One disk for data
- One disk for log
- Backup of data on another disk (often cheaper storage i.e. HDD)
- On crash:
 - If we have Data + Log; recovery succeeds
 - If we have Backup + Log; recovery succeeds
 - If we have Data + Backup (No Log); recovery fails

High Availability

- Service is available even during a crash
- Utilize failover server
 - A backup server containing all the data + log
 - In case main server crashes, failover takes over (delay due to redoing latest changes)
 - Takes over until main server crash has been fixed
- Availability measured in 9s
 - 99.9%, 99.99%, 99.999%,... uptime monthly / yearly
 - Large data companies boast five 9s (99.999%) or higher
 - Snowflake has five 9s, but depending on the provider and region you are in this could get down to three 9s

How to implement Transactions?

- Consistency ~= satisfying constraints
 - Use indexes for primary and foreign key, check statements, ...
- Atomicity and Durability = tracking changes
 - Logging "before" values to UNDO changes
 - Logging "after" values to REDO changes
- Isolation = preventing corrupting changes
 - Pessimistic: Locking to prevent conflicts
 - Optimistic: Time stamps to detect conflicts

Isolation and Serializability

- Want transactions to satisfy serializability:
 - The state of the database should always look as if the committed transaction ran in some serial schedule
- The scheduler of the DBMS is allowed to choose the order of transactions
 - It is not necessarily the transaction that is started first, which is first in the serial schedule

A Simple Scheduler

- A simple scheduler maintains a queue of transactions and carry them out in order
- Problems:
 - Transactions must *wait* for each other, even if unrelated
 - Some transactions may take very long
 - Possibly smaller throughput (Why?)
 - Wasted resources and longer wait time

Interleaving Schedulers

- Most databases have schedulers that allow the actions of transactions to interleave
- However, the result should be as if some serial schedule was used
 - A non-serial schedule that yields the same outcome as a serial schedule is known as a serializable schedule
- We will now study a mechanism that enforces "serializability": Locking
- Other methods exist: Time stamping / optimistic concurrency control
 - Out of scope for this course

Locks

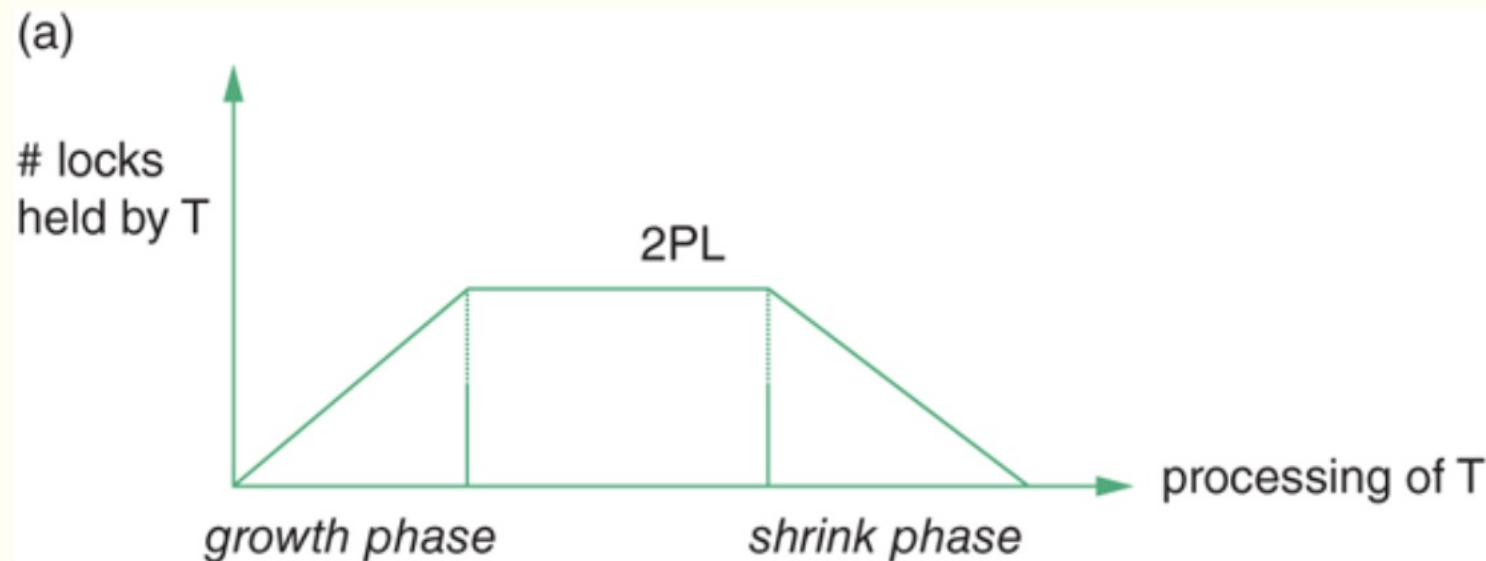
- In its simplest form, a lock is a right to perform operations on a database element
- Only one transaction may hold a lock on an element at any time
- Locks must be requested by transactions and granted by the locking scheduler
- Typically, two types of locks: Read/Shared (S) and Write/Exclusive (X)
 - If T1 has an (S) lock on resource A, and T2 wants an (S) lock on A, it is granted
 - Now, T3 requests an (X) lock on A and gets queued, followed by T4 wanting an (S) lock on A
 - Should T4 be allowed to skip ahead of T3?
 - Fairness and avoiding starvation
 - Upgrade requests
 - T1 requests to upgrade its (S) lock to an (X) lock, skip ahead and await T2 to finish

Two-Phase Locking

- Growing Phase: Acquire locks
 - When reading from a database resource, get a shared (S) lock
 - When writing to a database resource, get an exclusive (X) lock
- Shrinking Phase: No new locks can be acquired, and start releasing locks
- Multiple variations:
 - Static 2PL: All locks acquired before running the transaction
 - Strict 2PL (S2PL): Release only shared locks during shrinking phase
 - Rigorous 2PL (R2PL) / Strong Strict 2PL (SS2PL): Release locks at COMMIT / ABORT
- Commonly implemented, since:
 - Simple to understand and works well in practice
 - It makes transaction rollback easier to implement
 - But: Optimistic approaches are becoming more popular

Strict Two-Phase Locking

- Strict 2PL protocol:
 - Before reading a record/page, get a shared (S) lock
 - Before writing a record/page, get an exclusive (X) lock
 - A record/page cannot have an (X) lock at the same time as any other lock
 - Release (S) locks during shrinking phase and (X) locks on COMMIT/ABORT/ROLLBACK



Rigorous Two-Phase Locking

- Rigorous 2PL protocol:
 - Before reading a record/page, get a shared (S) lock
 - Before writing a record/page, get an exclusive (X) lock
 - A record/page cannot have an (X) lock at the same time as any other lock
 - Release all locks on COMMIT/ABORT



Locks and Deadlocks

- The DBMS sometimes must make a transaction wait for another transaction to release a lock
- This can lead to deadlock if e.g. A waits for B, and B waits for A
- In general, we have a deadlock exactly when there is a cycle in the waits-for graph
- Deadlocks are resolved by aborting some transaction involved in the cycle

Exercise

- Consider the following schedule of three transactions, T1, T2, and T3
- T1.read(A); T2.read(B); T3.write(C); T3.write(B); T1.write(B); T2.read(A); T3.write(A);
- Will this result in a deadlock?
 - Yes
- Will any of the transactions finish?
 - T2

Avoiding Deadlocks

- Upgrade requests can also deadlock

<pre>SELECT :x=counter FROM table WHERE <condition> :x=:x+1 UPDATE table SET counter = :x WHERE <condition></pre>	<pre>UPDATE table SET counter = counter+1 WHERE <condition> SELECT :x=counter FROM table WHERE <condition></pre>	<pre>SELECT FOR UPDATE :x=counter FROM table WHERE <condition> :x=:x+1 UPDATE table SET counter = :x WHERE <condition></pre>
---	---	--

- Order Matters:
 - With consistent order of access, deadlocks are avoided
 - Optimizer may not allow control over order!

Phantom Tuples

- Suppose we lock tuples/rows where $A=1$ in a relation and another transaction inserts a tuple with $A=1$
 - For some transactions this may result in unserializable behaviour
 - As in, it will be clear that the tuple/row was inserted during the course of a transaction.
- Such tuples/rows are called phantoms.

Phantom Example

```
CREATE TABLE B (x INTEGER, y INTEGER);
```

```
INSERT INTO B VALUES (1, 2);
```

```
BEGIN TRANSACTION;
```

```
SELECT MIN(y) FROM B WHERE x = 1;
```

```
-- Another transaction
```

```
BEGIN TRANSACTION;
```

```
INSERT INTO B VALUES (1, 1);
```

```
COMMIT;
```

```
-- Repeat the SAME read!
```

```
SELECT MIN(y) FROM B WHERE x = 1;
```

```
COMMIT;
```

Avoiding Phantoms

- Phantoms can be avoided by putting an (X) lock on a relation before adding tuples.
 - However, this leads to poor concurrent performance
- A technique called “index locking” can be used to prevent other transactions from inserting phantom tuples but allow most non-phantom insertions
- In SQL, the programmer may choose to either allow phantoms in a transaction or insist they should not occur.

Isolation Levels in Modern Systems

- READ UNCOMMITTED
 - a transaction can read uncommitted changes
- READ COMMITTED
 - a transaction only reads committed data, some other transaction may overwrite this data
- REPEATABLE READ
 - a transaction only reads committed data, other transactions cannot overwrite this data, but phantoms are possible
- SERIALIZABLE
 - ensures serializable schedule with no anomalies

May violate the "I" in ACID

Ensures the "I" in ACID

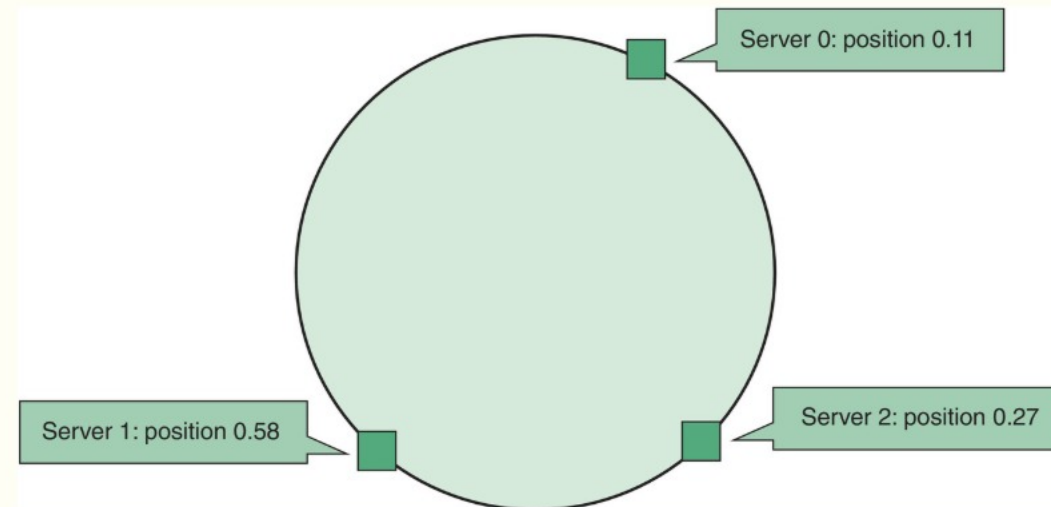
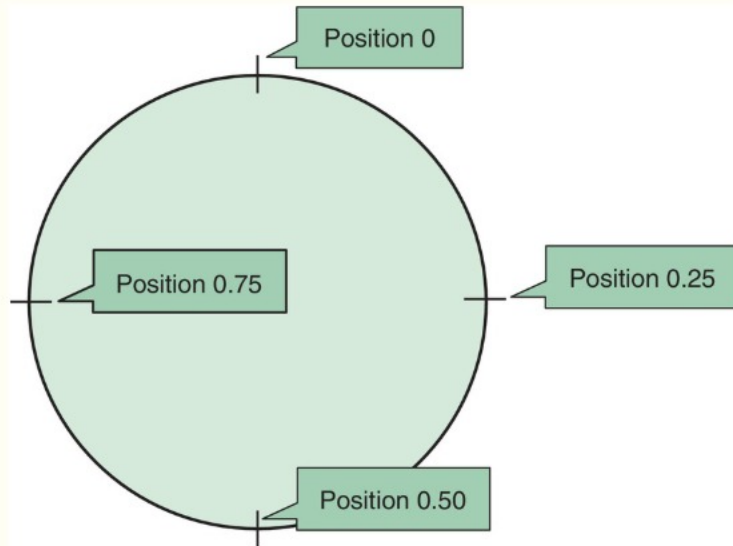
Distributed Storage Systems

Goals of Distributed Storage

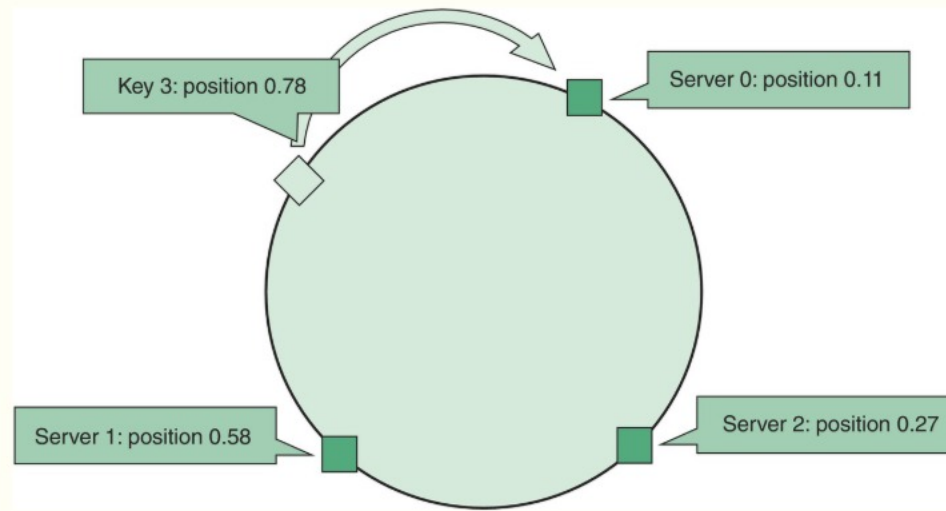
- Workload sharing – balance
 - Increased performance
 - Better resource utilization
- Redundancy (replicas)
 - Failure handling
 - High Availability

Load balancing using Consistent Hashing

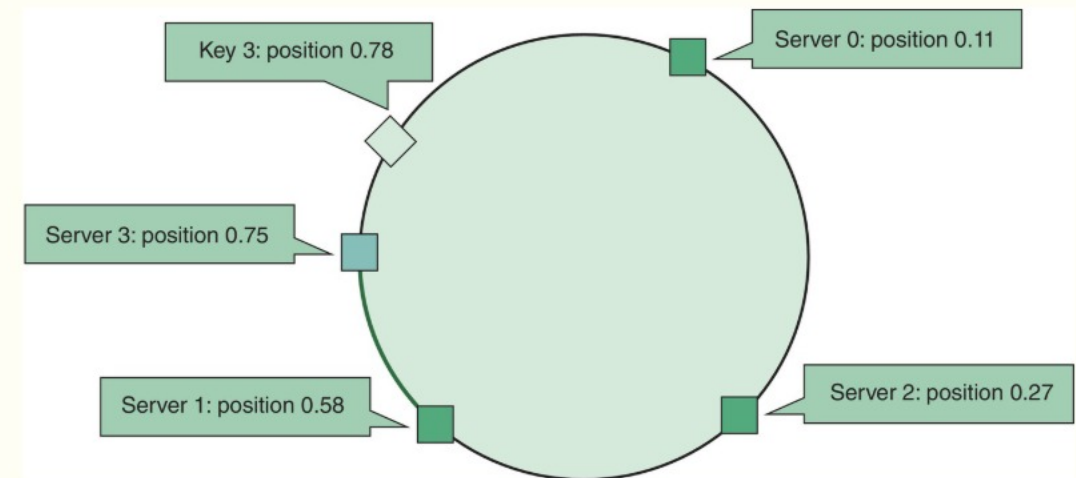
- Assign servers into a hash table
- Assign data to servers using the hash table
- What if we want to remove a server or add a new server?
 - Redistribute most of the data
- Consistent Hashing (ring-topology)



Load balancing using Consistent Hashing



Adding data:
Find position on ring (hash value)
Add to the closest server (clockwise)



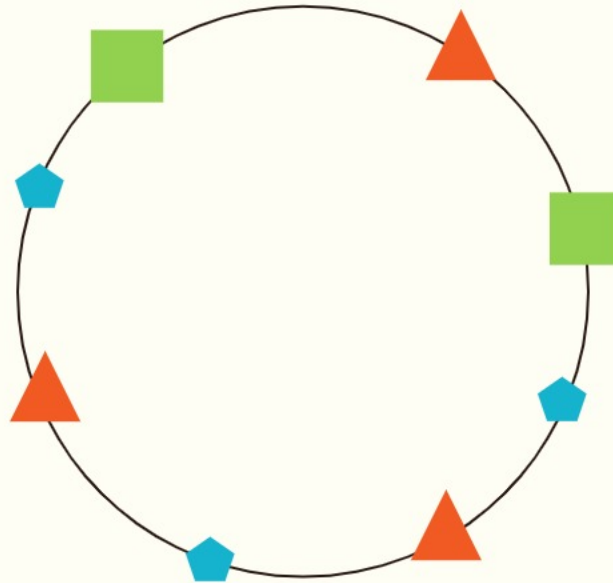
Adding Server 3:
Only redistribute data from Server 0

Consistent Hashing: Problems

- Random assignments can lead to skewed distributions
- Does not reflect a server's capabilities
 - Not all servers have the same hardware
- Adding a new server creates high load for 1 server

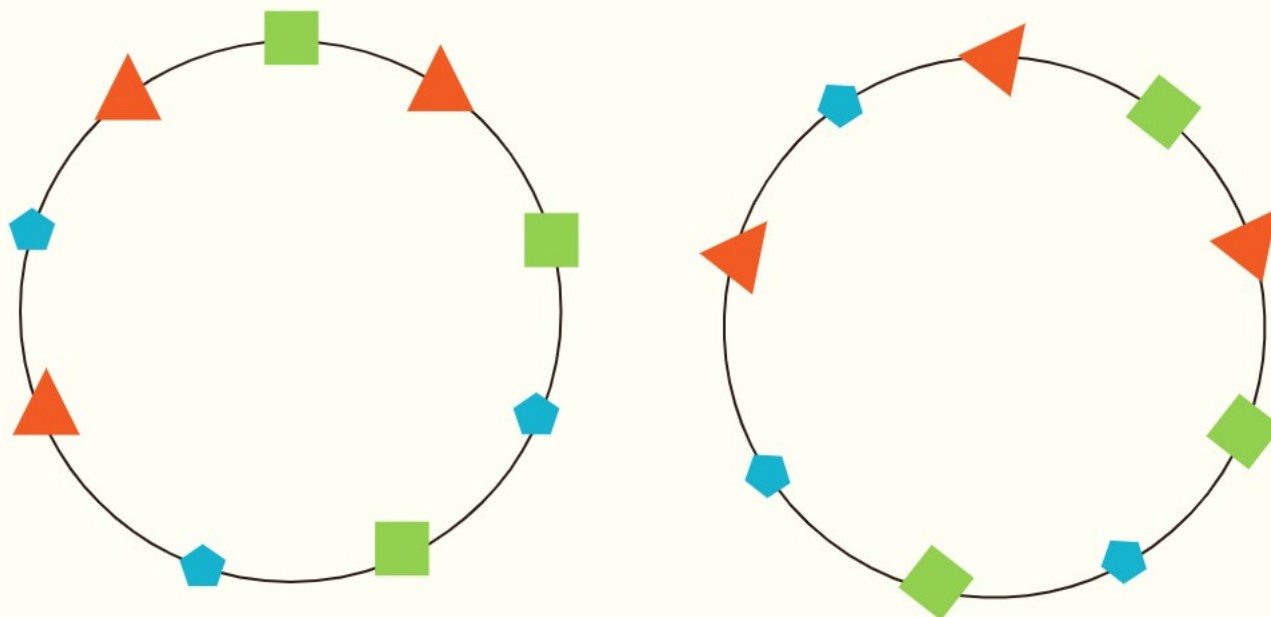
Consistent Hashing: Virtual Servers

- 1 Server = multiple virtual servers
- Assign virtual servers to the ring instead of the singular server
- Amount of virtual servers can reflect capabilities
 - Better computing and storage resources = more virtual servers



Consistent Hashing: Redundancy

- The consistent hashing scheme can also be used for durability
- Backup data from one (virtual) server to another
- Server crash
 - High Availability: Forward request to backup (virtual) servers
 - Recovery: Once it is back up use the backup servers to follow recovery protocol
- Rotation
 - Same ring
 - Rotation distributes data accordingly



Replica Consistency

- Sequential (or strong) consistency: All updates are seen by all processes in the same order. As a result, the effects of an update are seen by all observers. There is no inconsistency.
 - Roughly the same as Isolation in ACID
- Weak consistency: Observers might see inconsistencies among replicas
- Eventual consistency: A form of weak consistency, where at some point, in case there is no failure, all replicas will reflect the last update.

Tunable Consistency

- Not a binary system
 - N replicas, R read quorum, W write quorum
- $R = W = 1$ gives eventual consistency
- $R + W > N$ gives strong consistency
- Example:

If the replication factor is 3, then the consistency level of the reads and writes combined must be at least 4 to achieve strong consistency.

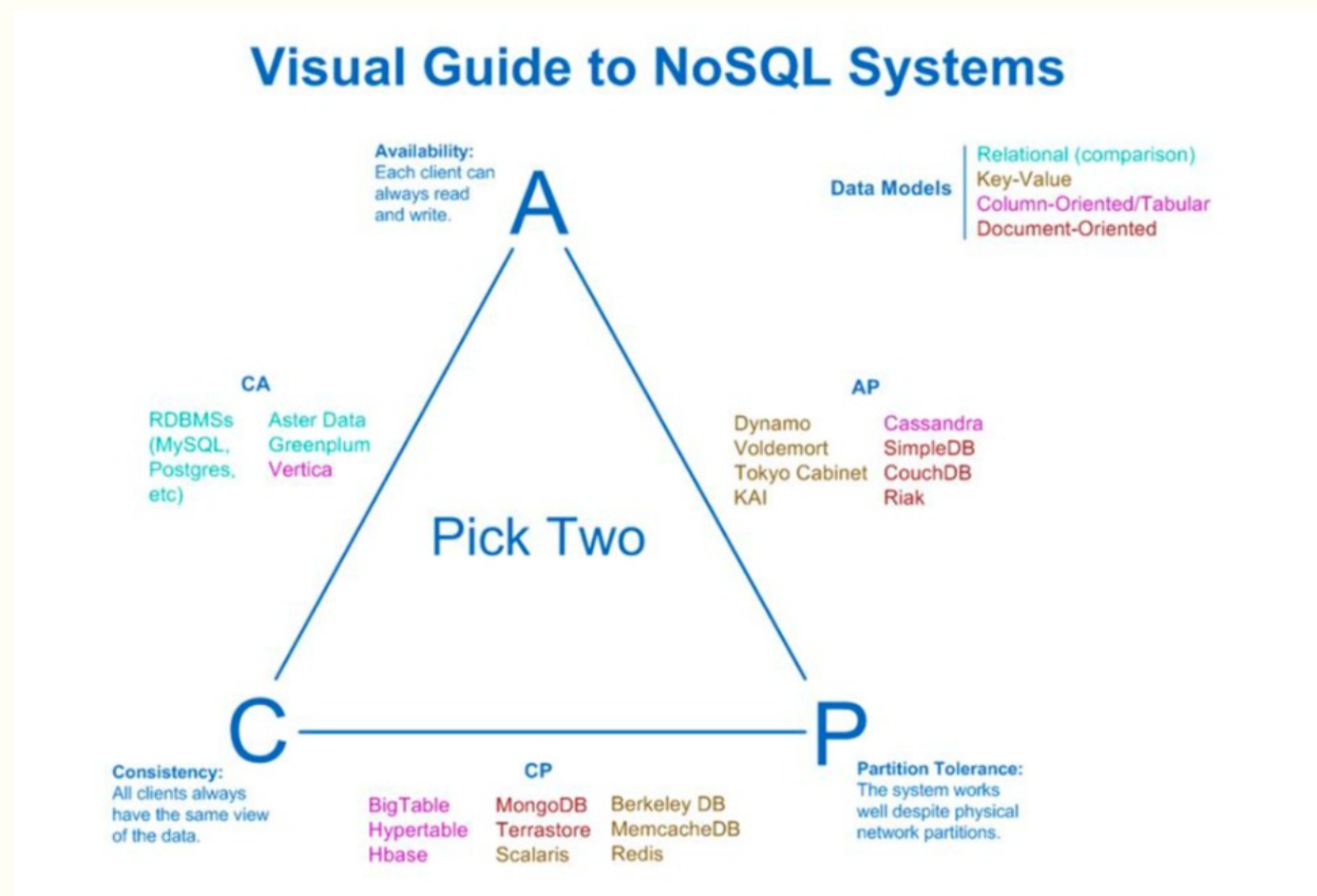
As in, read operations has to read from 2 out of 3 replicas to verify the value and write operations has to update 2 out of 3 replicas to successfully write

CAP Theorem

- C = Consistency
 - Readers read the most recent update
- A = Availability
 - A valid answer is returned, even if one or more nodes are down
- P = Partition Tolerance
 - Partition: The network becomes disconnected
 - A distributed system works despite the network failure

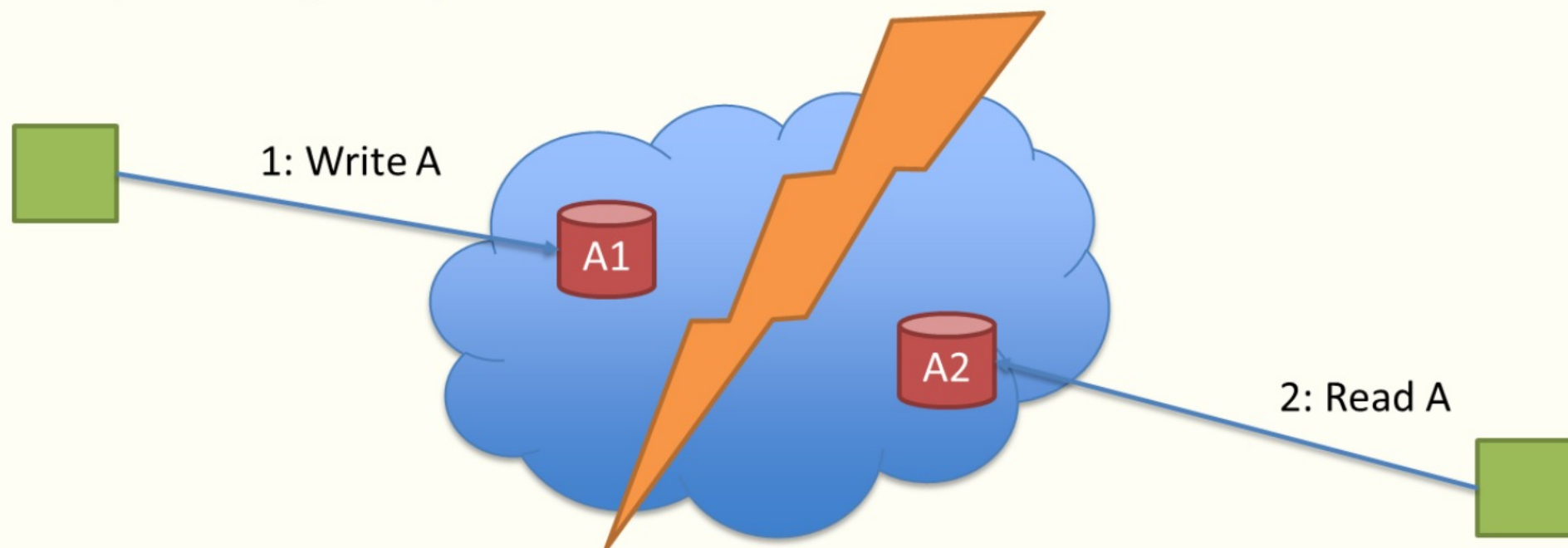
Incorrect (but typical) formulation

- You can only get **two** of **Consistency**, **Availability**, and **Partition Tolerance**



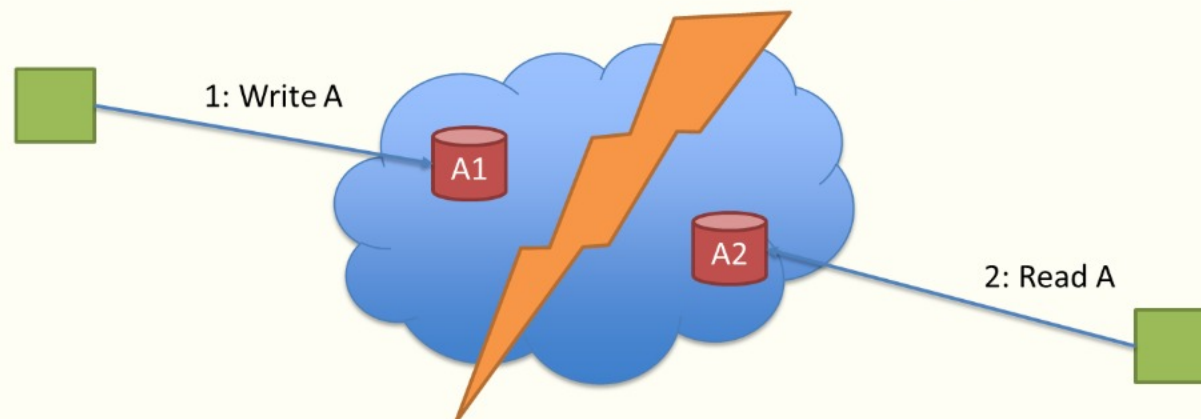
Correct (but useless) formulation

- In a **partitioned** network, choose between **Consistency** and **Availability**
- Proof: Simple thought experiment

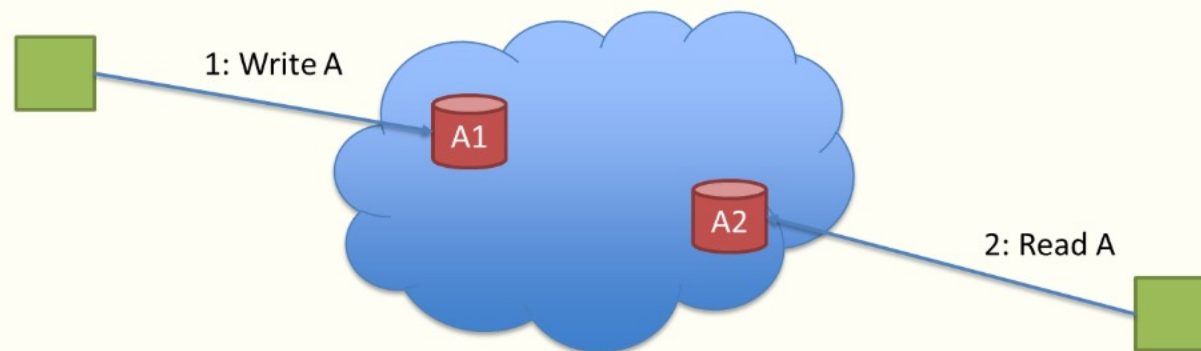


PACELC (useful!) formulation

- In a **partitioned** network, choose between **Availability** and **Consistency**



- Else** (regular operation), choose between **Latency** and **Consistency**



Takeaways!

- Transactions / ACID:
 - Atomicity:
 - All or nothing
 - Write-Ahead Logging (WAL): UNDO
 - Consistency: Constraints / Data Validation
 - Isolation:
 - Locking (Pessimistic): 2PL / S2PL / R2PL (SS2PL)
 - Isolation levels: Read Uncommitted / Read Committed / Repeatable Read / Serializable
 - Durability:
 - Backups and recovery protocols
 - Write-Ahead Logging (WAL): REDO
- Distributed Systems
 - Workload sharing: Load balancing using consistent hashing
 - Redundancy: High availability and durability
 - Choose between **Availability** and **Consistency** during network partitions, otherwise between **Latency** and **Consistency**